

清华大学电子工程系本科生核心课

数据与算法 - 第9讲

排序

冯伟 (fengwei@tsinghua.edu.cn)

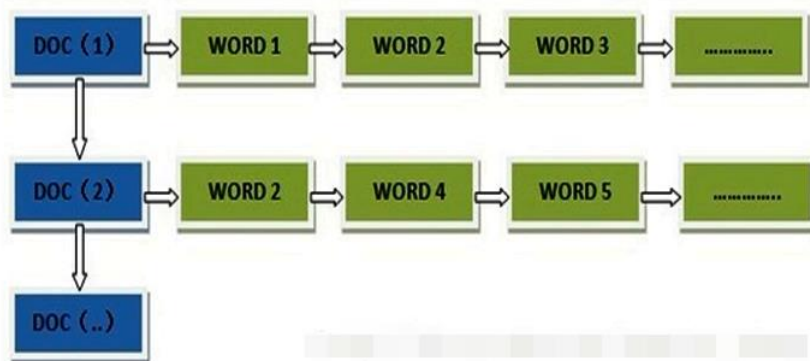
2024年11月12日



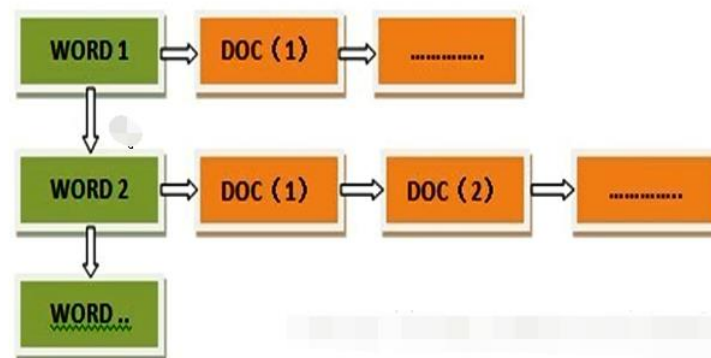


从查找到排序：佩奇的故事

- 索引结构 ($i = g(k)$) $\rightarrow$ 索引文件
- 倒排文件：通过关键字快速获取文件集合
- 百度和谷歌：利用倒排索引实现从关键字到互联网文件的快速查找



正向索引：从文件到关键字



倒排索引：从关键字到文件
(搜索引擎广泛使用)



从查找到排序：佩奇的故事

- 有了倒排索引，我们可以快速得到搜索结果，但是怎么决定搜索结果的优先级呢？
- 这时候就用到了的一种叫PageRank（佩奇排名）的算法：根据网页之间相互的链接关系（链接体现价值），计算网页的权重，按照权重进行排序，权重越高排得越靠前



- 1996年发明算法
- 1998年申请专利，创立谷歌
- 2024年2月，总市值全球排名第四



提纲

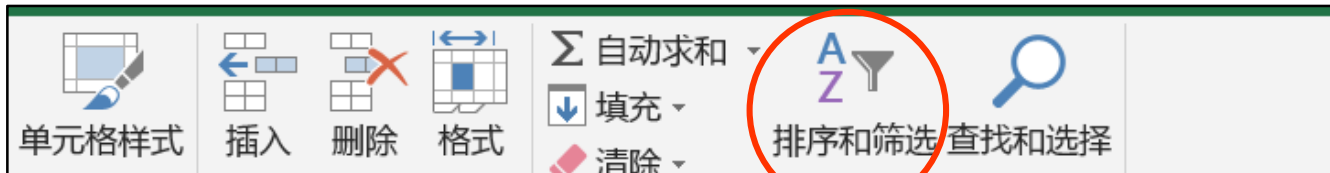
- 基本概念
- 基本排序方法
- 希尔排序
- 快速排序
- 归并排序
- 堆排序



提纲

- 基本概念
- 基本排序方法
- 希尔排序
- 快速排序
- 归并排序
- 堆排序

排序在计算机数据处理中很常见



对区域或表中的数据进行排序

对数据进行排序是数据分析不可缺少的组成部分。你可能需要执行以下操作：将名称列表按字母顺序排列；按从高到低的顺序编制产品存货水平列表，或按颜色或图标对行进行排序。对数据进行排序有助于快速直观地显示数据并更好地理解数据，有助于组织并查找所需数据，有助于最终做出更有效的决策。



在日常数据处理中，约有1/4的时间用在排序上





元素与关键字

- 排序：将一组杂乱无章的数据按一定的规律顺次排列
- 数据表（DataList）：待排序数据对象的有限集合
- 关键字（Key）：多个属性域中可以用来区分对象进而作为排序依据的域，也称**关键码**
 - 关键字不同，排序唯一
 - 关键字有相同的，排序可能不唯一



排序

- 排序： n 个对象的序列为 $\{R_{[0]}, R_{[1]}, \dots, R_{[n-1]}\}$ ，对应关键码为 $\{K_{[0]}, K_{[1]}, \dots, K_{[n-1]}\}$ ，确定一种排列 $p_{[0]}, p_{[1]}, \dots, p_{[n-1]}$ 使各关键码满足
 - 非递减关系： $K_{[p_{[0]}]} \leq K_{[p_{[1]}]} \leq \dots \leq K_{[p_{[n-1]}]}$
 - 或非递增关系： $K_{[p_{[0]}]} \geq K_{[p_{[1]}]} \geq \dots \geq K_{[p_{[n-1]}]}$
- 根据关键码递增或递减的顺序，把数据对象依次排列起来，使一组任意排列的对象变成一组按其关键码线性有序的对象



逆序

- 逆序：待排序列中如果有两个元素顺序不符合排序要求，则称为一个逆序，也叫倒置
 - 逆序的数目可以描述当前序列和有序序列的差距
 - 有序序列：逆序 = 0
 - 反序序列：逆序数目达到最大
 - 对于一些排序算法，逆序数目直接决定了排序算法的时间复杂度



排序算法的稳定性

- 如果在对象序列中有对象 $R_{[i]}$ 和 $R_{[j]}$ ，其关键码 $K_{[i]} = K_{[j]}$ ，且在排序之前， $R_{[i]}$ 排在 $R_{[j]}$ 前面；若排序之后， $R_{[i]}$ 仍在 $R_{[j]}$ 之前，则排序是稳定的，否则称为不稳定的
 - 结果有差异，不稳定不一定不好，具体应用具体考虑
- 根据在排序过程中数据元素是否完全存放在内存中
 - 内排序：在排序期间数据对象全部存放在内存的排序
 - 外排序：数据元素过大，排序过程中不断在内、外存之间移动元素



排序算法的基本操作

➤ 交换

```
template<class T> //基于模板定义
inline void exch(T &e1, T &e2){
    T tmp;    //临时变量
    tmp = e1;
    e1 = e2;
    e2 = tmp;
}
```

➤ 比较 - 交换

```
template<class T>
void compExch(T &e1, T&e2) {
    if (e1 > e2)    exch(e1, e2);
}
```

基本排序算法由一系列的“比较-交换”操作组成

扩展思考：也有不基于比较的排序算法，如计数排序、桶排序等



排序算法性能评估

➤ 执行时间

- 数据比较次数、数据移动次数
- 一般都按平均情况进行估算，受初始排列及对象个数影响较大的，按最好和最坏情况进行估算

➤ 算法执行时所需的附加存储（额外内存空间开销）

- 除了使用规模很小的堆栈或者表外，不需要使用任何额外内存操作
- 使用链表和指针、数组索引来代表数据，存储指针或索引需额外内存
- 需要额外的空间来存储要排序的元素序列的副本



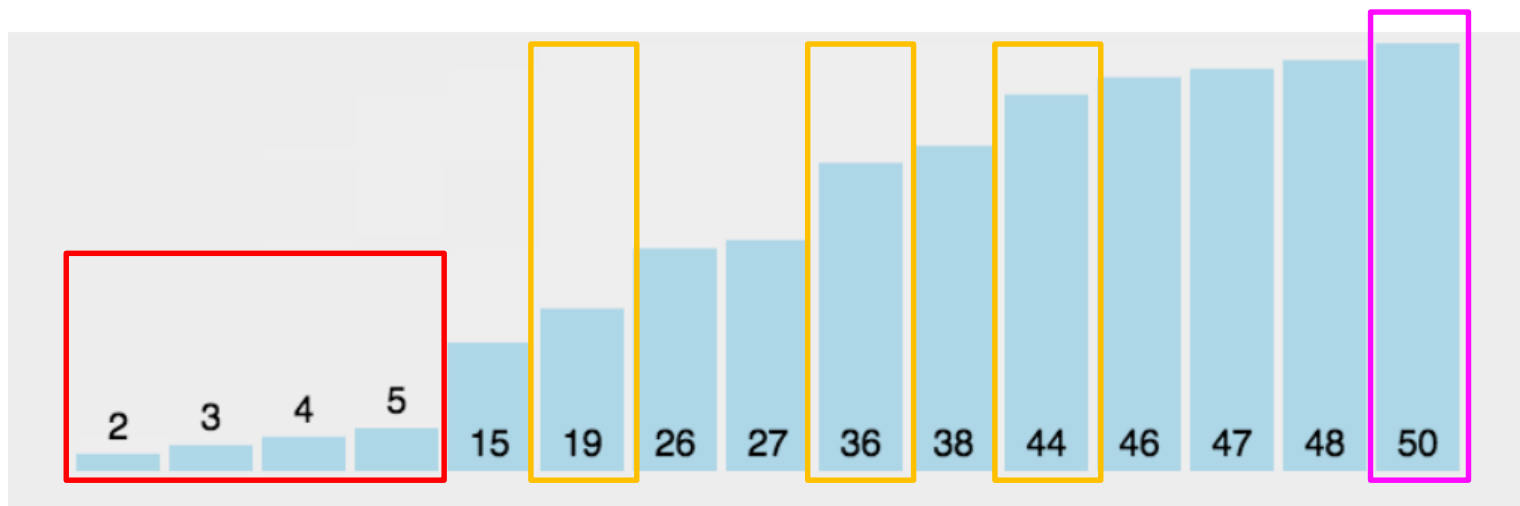
提纲

- 基本概念
- 基本排序方法
- 希尔排序
- 快速排序
- 归并排序
- 堆排序



基本排序方法

基本 → 思想朴素、操作直观



特征 ①：逆序数 = 0

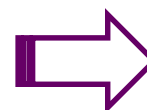
特征 ②：子序列有序

特征 ③：各就各位

逐对排逆

逐个入序

逐位确认



冒泡排序

插入排序

选择排序



冒泡排序

设待排序序列中的元素个数为 n ,首先比较第 $n-1$ (0) 个、即序列中最后一个元素和第 $n-2$ (1) 个元素, 如果存在逆序, 即将这两个元素交换; 然后对第 $n-2$ (1) 个和第 $n-3$ (2) 个元素 (可能是刚交换过来的元素) 做同样处理; 重复此过程直到处理完第 1 ($n-2$) 个和第 0 ($n-1$) 元素, 称为一趟冒泡; 这样最多做 $n-1$ 趟冒泡就能把所有元素排好序。

5	3	8	4	6
5	3	8	4	6
5	3	4	8	6
5	3	4	8	6
3	5	4	8	6



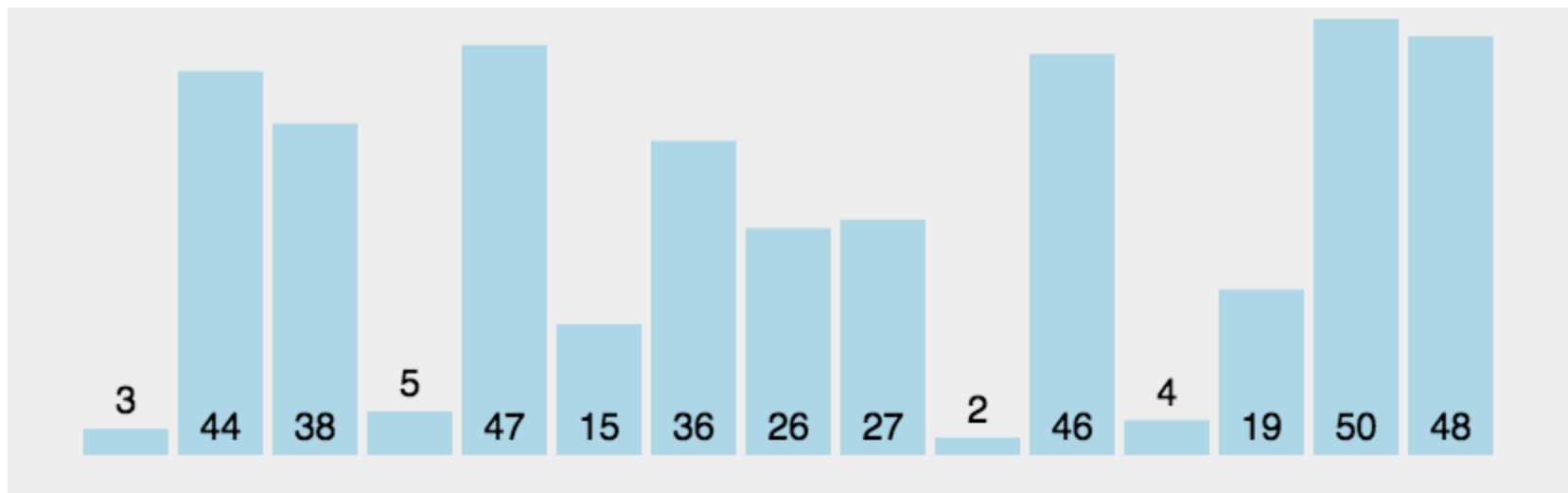
冒泡排序

5	3	8	4	6
3	5	4	8	6
3	4	5	6	8
3	4	5	6	8
3	4	5	6	8
3	4	5	6	8



冒泡排序 - 动画示意

设待排序序列中的元素个数为 n ,首先比较第 $n-1$ (0) 个、即序列中最后一个元素和第 $n-2$ (1) 个元素, 如果存在逆序, 即将这两个元素交换; 然后对第 $n-2$ (1) 个和第 $n-3$ (2) 个元素 (可能是刚交换过来的元素) 做同样处理; 重复此过程直到处理完第 1 ($n-2$) 个和第 0 ($n-1$) 元素, 称为一趟冒泡; 这样最多做 $n-1$ 趟冒泡就能把所有元素排好序。





冒泡排序 - 实现

```
template<class T>
void bubble_sort(T *a, int l, int r) {
    int i,j;
    for(i = l; i < r; i++) {
        for(j = r; j > i; j--)
            if(a[j-1] > a[j]) exch(a[j-1], a[j]); //发现逆序，就进行交换操作
        }
    }
}
```



冒泡排序 - 评价

➤ 比较操作的次数：

$$(n-1) + (n-2) + \cdots + 2 + 1 = \frac{n(n-1)}{2}$$

➤ 最坏情况交换操作次数：

$$(n-1) + (n-2) + \cdots + 2 + 1 = \frac{n(n-1)}{2}$$

➤ 效率与输入序列特性无关

➤ 不论交换操作的执行次数，冒泡排序时间复杂度 $O(n^2)$



冒泡排序 - 改进

- 一些序列不需要进行全部冒泡过程
 - 设置标志, 如果没有交换操作, 则无逆序, 排序结束
- 改进后的算法
 - 最好情况 $n - 1$ 次比较, 0次交换
 - 一般和最坏情况, 复杂度仍是 $O(n^2)$
- 冒泡排序是稳定的
 - 比较和交换只在相邻元素间进行, 关键字相等的元素不会交换

插入排序

- **逐个入序**：每步将待排的元素，按其关键字插入到前面已经排好序的一组元素（子序列）中的适当位置，**保持这组元素有序**；当所有元素都完成插入，则整个序列必然有序

“从一个子序列长起来”

- ## ➤ 依找到插入位置的方法
- 直接插入排序
 - 折半插入排序

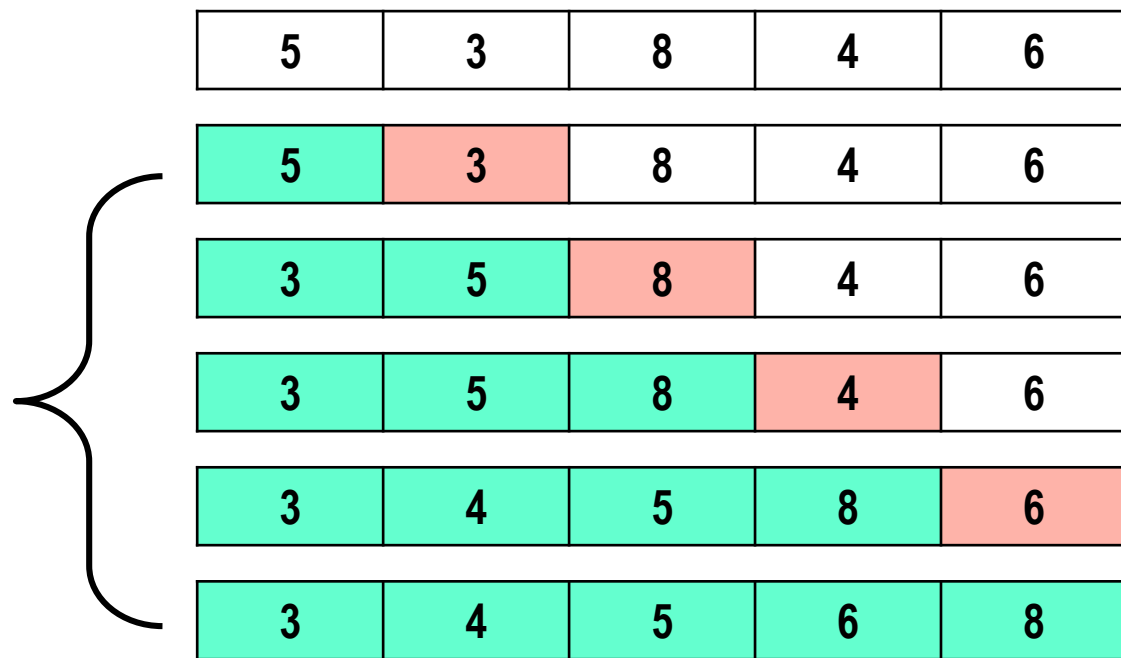




直接插入排序

- 插入第 $i + 1$ 个元素时，前面 $a[0], a[1], \dots, a[i - 1]$ 已经排好
- 比较 $a[i]$ 与 $a[i - 1], a[i - 2], \dots$ ，找到位置插入 $a[i]$ ，原来元素向后顺移

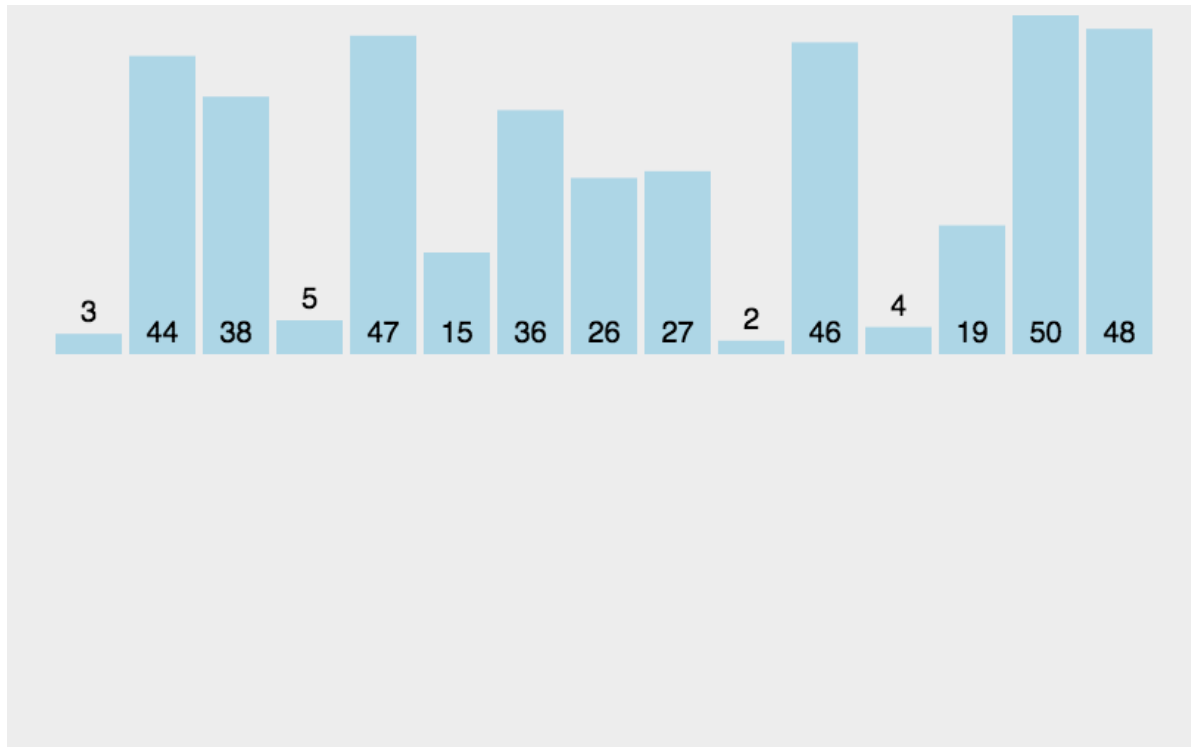
绿色表示前面已经排好的子序列





直接插入排序 – 动画示意

- 插入第 $i + 1$ 个元素时，前面 $a[0], a[1], \dots, a[i - 1]$ 已经排好
- 比较 $a[i]$ 与 $a[i - 1], a[i - 2], \dots$ ，找到位置插入 $a[i]$ ，原来元素向后顺移





直接插入排序 - 实现

```
template<class T> void insert_sort(T *a, int l, int r) {  
    int    i, j;  
    T      tTag;  
    for(i = r; i > l; i--)  
        if(a[i-1] > a[i]) exch(a[i-1], a[i]); //序列最小元素放到第一位, 防止  
    for(i = l+2; i <= r; i++){ //出现数组下标小于0的情况 (负溢)  
        j = i; tTag = a[i]; //保存待插入元素  
        while(tTag < a[j-1]) //不是插入位置  
            {a[j] = a[j-1]; j--;} //元素向后移位  
        a[j] = tTag; //插入位置  
    }  
}
```




直接插入排序 - 评价

- 直接插入排序的时间与待排序列的初始排列有关
 - 最好情况：已经有序，比较 $n - 1$ 次，元素移动 $2(n - 1)$ 次
 - 最坏情况：完全逆序，比较 $n(n - 1)/2$ 次，移动 $n(n - 1)/2$ 次
 - 平均情况：比较 $n^2/4$ 次，移动 $n^2/4$ 次
- 时间复杂度 $O(n^2)$
- 直接插入排序是稳定的
 - 待插入元素从后向前逐个比较，一旦当前元素关键字小于等于待插元素，待插元素会变成当前元素的直接后继，所以关键字相同的元素顺序不会改变



折半插入排序 - 利用前序列的有序性

```
template<class T>void insert_bisort(T *a, int l, int r){  
    int      i, j, right, left, middle;  
    T        e;  
    for(i = l+1; i <= r; i ++){  
        e = a[i]; left = l; right = i-1;           //查找范围  
        while(left <= right){                      //折半查找  
            middle = (left + right)/2;              //中间位置  
            if(e < a[middle]) right = middle-1;     //向左缩小区间  
            else left = middle+1;                   //否则, 向右缩小区间  
        }  
        for(j = i-1; j >= left; j--) a[j+1] = a[j]; //成块移动, 空出插入位置  
        a[left] = e;                                //插入  
    }  
}
```



折半插入排序 - 评价

- 长度为 k 的序列，折半查找约需 $\lfloor \log_2 k \rfloor + 1$ 次比较（与初始排列无关）， n 个元素的序列需要比较次数约为 $n \log_2 n$
- 最坏和平均情况下的比较次数要好于直接插入排序，但在最好情况下不如直接插入排序；在元素初始排列有序或接近有序的时候，直接插入效率更高
- 元素移动次数与直接插入排序相同
 - 总时间复杂度 $O(n^2)$
 - 比较次数更少，元素移动次数与直接插入排序相同，依赖元素的初始排列



选择排序

逐位确认 → 最小的在第一个、次小、次次小

- 在 $a[i] \sim a[n - 1]$ 中选择最小的元素
- 若不是其中的第一个元素，则与第一个元素交换
- 在 $a[i + 1] \sim a[n - 1]$ 中重复上述步骤，直到仅余一个元素

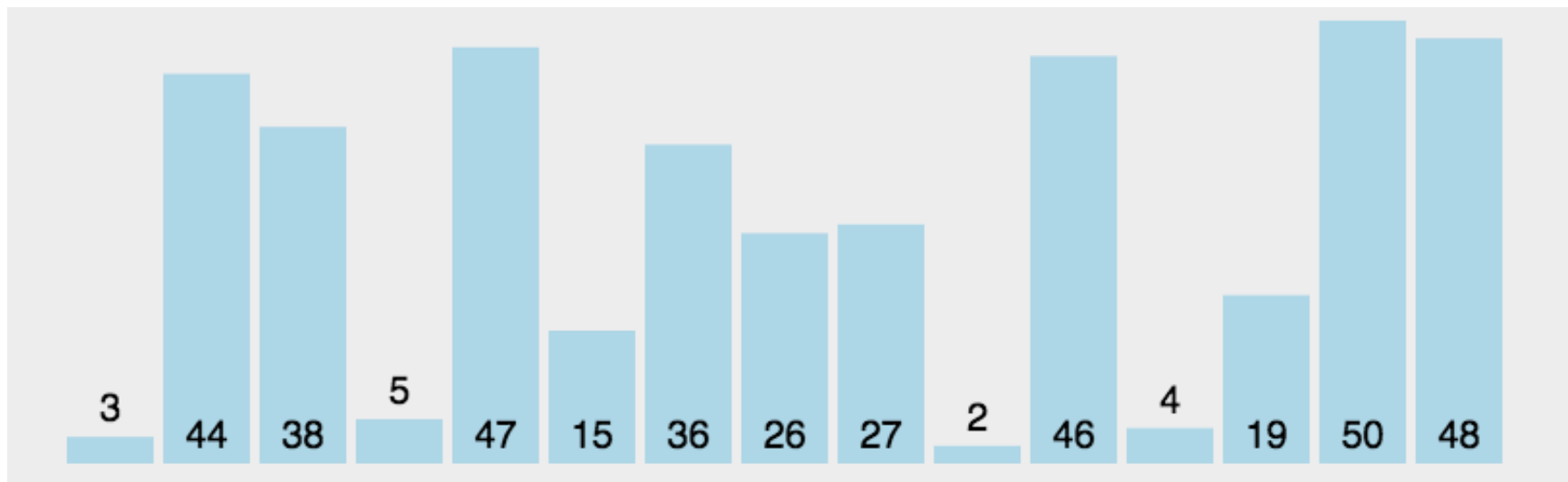
5	3	8	4	6
5	3	8	4	6
3	5	8	4	6



选择排序 – 动画示意

逐位确认 → 最小的在第一个、次小、次次小

- 在 $a[i] \sim a[n - 1]$ 中选择最小的元素
- 若不是其中的第一个元素，则与第一个元素交换
- 在 $a[i + 1] \sim a[n - 1]$ 中重复上述步骤，直到仅余一个元素





选择排序 - 实现

```
template<class T>void select_sort(T *a, int l, int r){  
    int i, j, min;  
    for(i = l; i < r ; i ++){  
        min = i;  
        for(j = i+1; j <= r ; j++){  
            if(a[j] < a[min]) min = j;  
            //选择序列最小元素  
            //交换操作  
            exch(a[i],a[min]);  
        }  
    }  
}
```



选择排序 - 评价

- 比较次数与序列初始排列无关，需要 $n(n - 1)/2$ 次比较操作
- 元素移动次数与待排序列初始排列有关： $0, n - 1$
- 优势：实现简单，执行时间比较固定
- 适合元素规模大，用于比较的关键字规模小的序列，比较代价小（仅关键字），移动代价大（整个元素）
 - 例如学生信息（姓名、学号、班级、成绩等）
- 不稳定
 - 非相邻元素交换，关键字相同元素的相对位置在排序中不能保证



选择排序 - 评价

5 (A)	6	5 (B)	2	3
2	6	5 (B)	5 (A)	3
2	3	5 (B)	5 (A)	6
2	3	5 (B)	5 (A)	6
2	3	5 (B)	5 (A)	6
2	3	5 (B)	5 (A)	6

➤ 不稳定

- 非相邻元素交换，关键字相同元素的相对位置在排序中不能保证



基本排序方法比较

排序方法	比较次数			移动次数			稳定性
	最好情况	最坏情况	平均	最好情况	最坏情况	平均	
直接插入排序	$O(n)$	$O(n^2)$	$n^2/4$	$O(n)$	$O(n^2)$	$n^2/4$	√
折半插入排序	$O(n\log_2 n)$			$O(n)$	$O(n^2)$		√
冒泡排序	$O(n)$	$O(n^2)$	$n^2/2$	0	$O(n^2)$	$n^2/2$	√
选择排序	$O(n^2)$			0	$O(n)$		×

- 平均时间复杂度都是 $O(n^2)$
- 直接插入排序对于规模很小（ ≤ 25 ）的待排序列非常有效；平均情况，直接插入排序要比冒泡排序快一倍；对于基本有序的序列，插入排序的性能要好于选择排序
- 选择排序的性能与待排序序列的初始特性基本无关；对于元素规模很大、而关键字项规模比较小的序列，选择排序具有更好的时间效率
- 选择排序是不稳定的，而插入排序和冒泡排序都是稳定的
- 基本排序方法除了一个辅助元素外，不需要其他额外内存



提纲

- 基本概念
- 基本排序方法
- 希尔排序
- 快速排序
- 归并排序
- 堆排序



希尔排序

“从一个子序列长起来” → “从多个子序列长起来”

➤ 从“宏观”到“微观”的调整

➤ h -排序：序列中间距为 h 的元素有序（联想Floyd算法的 $D_k[i][j]$ ）

10	30	60	12	42	93	68	85	80
----	----	----	----	----	----	----	----	----

➤ 希尔排序(Donald Shell 1959, 35岁, 2015仙逝, 缩小增量排序)

- 设计步长序列 $h_0, h_1, \dots, h_m, h_0 = 1$
- 第1步将序列变成 h_m -排序
- 第2步将序列变成 h_{m-1} -排序
- 第 $m + 1$ 步后, 排序完成 (h_0 -排序)



希尔排序

4 - 排序

80	93	60	12	42	30	68	85	10
10	30	60	12	42	93	68	85	80



序列较小

2 - 排序

10	30	60	12	42	93	68	85	80
10	12	42	30	60	85	68	93	80

1 - 排序

10	12	42	30	60	85	68	93	80
10	12	30	42	60	68	80	85	93



基本有序



希尔排序

- 希尔排序比基本排序算法性能好
 - 开始步长 h 较大，子序列中元素较少，排序速度较快
 - 步长 h 的值逐渐变小，子序列中元素增多，但此时已基本有序（前面一直在处理逆序），因此排序速度仍然很快
- 计算复杂度没有完整分析，Knuth以实验统计数据为基础，估计为 $n^{1.25} \sim 1.6n^{1.25}$ （ n 很大时，基于直接插入排序）
- 希尔排序的性能取决于步长序列的选择
 - 好序列：1 8 23 77 281 1073, $h_i = 4^{i+1} + 3 \times 2^i + 1$;
 - 坏序列：1 2 4 8 16 32 64, $h_i = 2^i$ ；直到最后一步奇数位与偶数位才比较，效率不高；本质上是对逆序的结构化处理问题



提纲

- 基本概念
- 基本排序方法
- 希尔排序
- 快速排序
- 归并排序
- 堆排序



快速排序 - 基本思路

- 快速排序算法被称为“二十世纪十大算法之一”，应用广泛（标准C++中的 `qsort` 排序函数）
- 霍尔（C. Hoare）在1960年提出（26岁），是一种分治策略的算法
- 取序列中某元素为划分元素，将整个序列划分为左、右两个子序列
 - 左侧元素全部小于等于划分元素
 - 右侧元素全部大于等于划分元素
 - 划分元素在正确的位置
 - 对子序列做同样的划分操作，每次都将一个元素放在正确位置
 - 所有元素都在正确位置，排序完成
 - 可以递归实现



快速排序 - 递归实现

```
template<class T>void quicksort(T *a, int l, int r){  
    if(r <= l) return;                //子序列长度小于1（终止条件）  
    int i = partition(a, l, r);        //序列的划分操作  
    quicksort(a, l, i-1);              //对前半部分递归调用排序操作  
    quicksort(a, i+1, r);              //对后半部分递归调用排序操作  
}
```

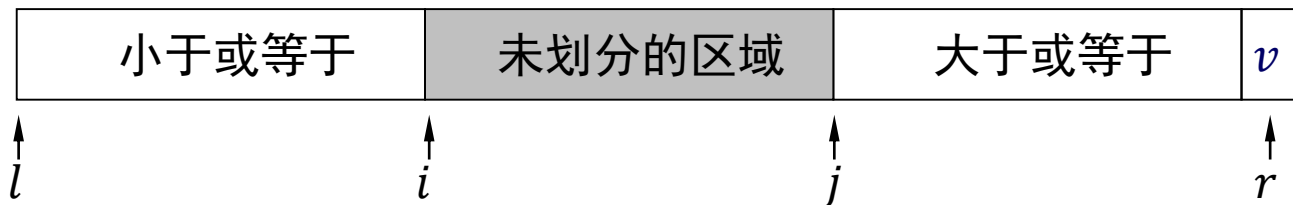
划分(partition)是快速排序的基本操作



快速排序 - 划分

l 和 r 为序列的第一个和最后一个元素， v 表示划分元素的值， i 表示左侧指针， j 表示右侧指针

- 选择 $a[r]$ 作为划分元素，使其划分后位于正确位置
- 从序列的最左边向中间扫描，找到一个比划分元素大的元素
- 从序列的最右边向中间扫描，找到一个比划分元素小的元素
- 大元素在左，小元素在右，则交换这两个元素
- 当扫描指针 i 和 j 相遇时（可能是划分元素），和划分元素交换，划分完成





快速排序 - 划分

```
template<class T>int partition(T *a, int l, int r) {  
    int      i = l-1, j = r;           //左右指针赋初值  
    T        e = a[r];                 //最右端元素为划分元素  
    while(1) {  
        while(a[++i] < e) ; if( i == r) break; //自左向右寻找, 包括划分元素  
        while(e < a[--j])    if( j == l) break; //自右向左寻找  
        if(i >= j) break;           //确定划分位置  
        exch(a[i],a[j]);            //交换元素  
    }  
    exch(a[i],a[r]);                //划分元素就位  
    return i;  
}
```



快速排序 - 举例

20	26	93	17	77	31	44	55	54
----	----	----	----	----	----	----	----	----

20	26	93	17	77	31	44	55	54
----	----	----	----	----	----	----	----	----

自左向右查找

20	26	93	17	77	31	44	55	54
----	----	----	----	----	----	----	----	----

自右向左查找

20	26	44	17	77	31	93	55	54
----	----	----	----	----	----	----	----	----

交换，自左向右查找

20	26	44	17	77	31	93	55	54
----	----	----	----	----	----	----	----	----

自右向左查找

20	26	44	17	31	77	93	55	54
----	----	----	----	----	----	----	----	----

交换，自左向右查找，相遇

20	26	44	17	31	54	93	55	77
----	----	----	----	----	----	----	----	----

交换



代表划分元素位置



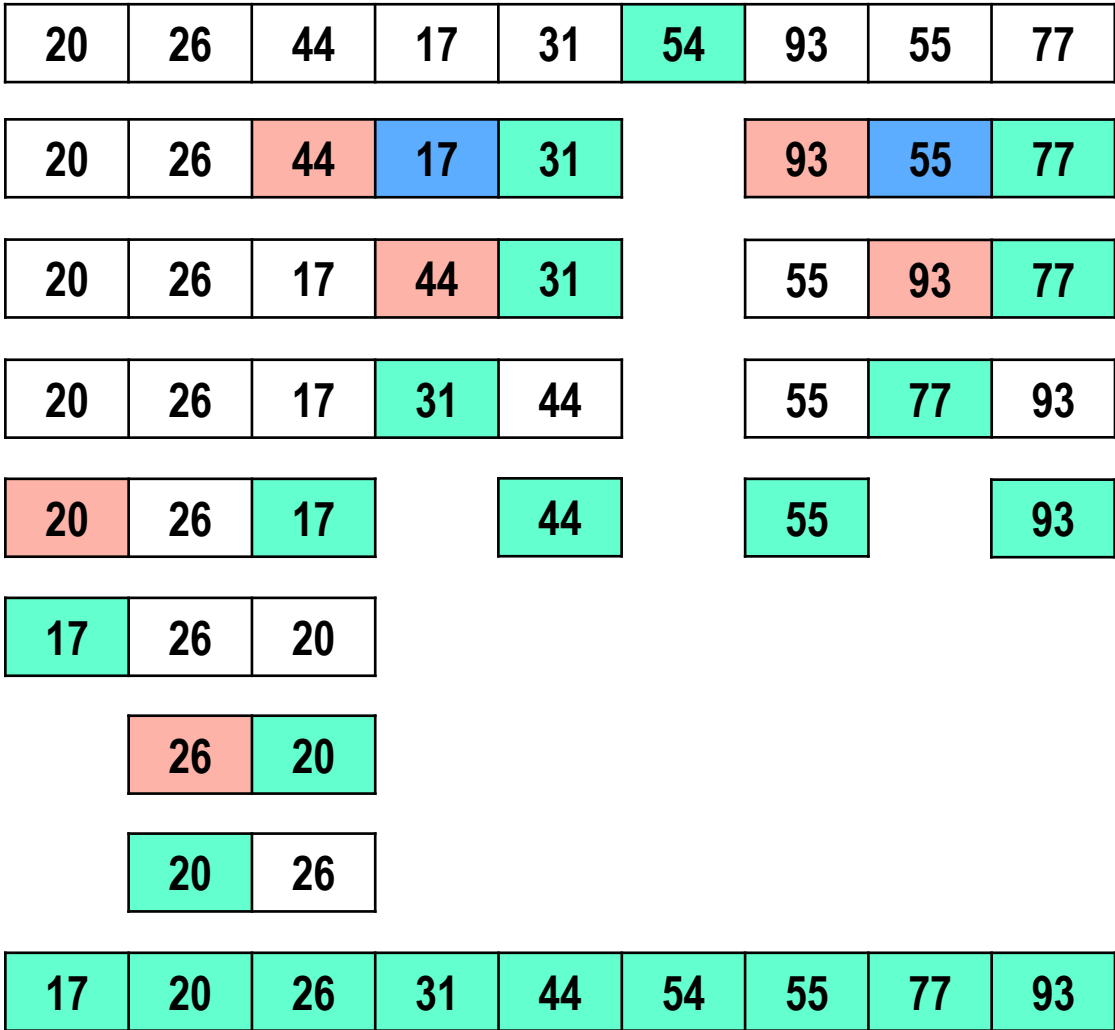
代表左指针位置



代表右指针位置



快速排序 - 举例

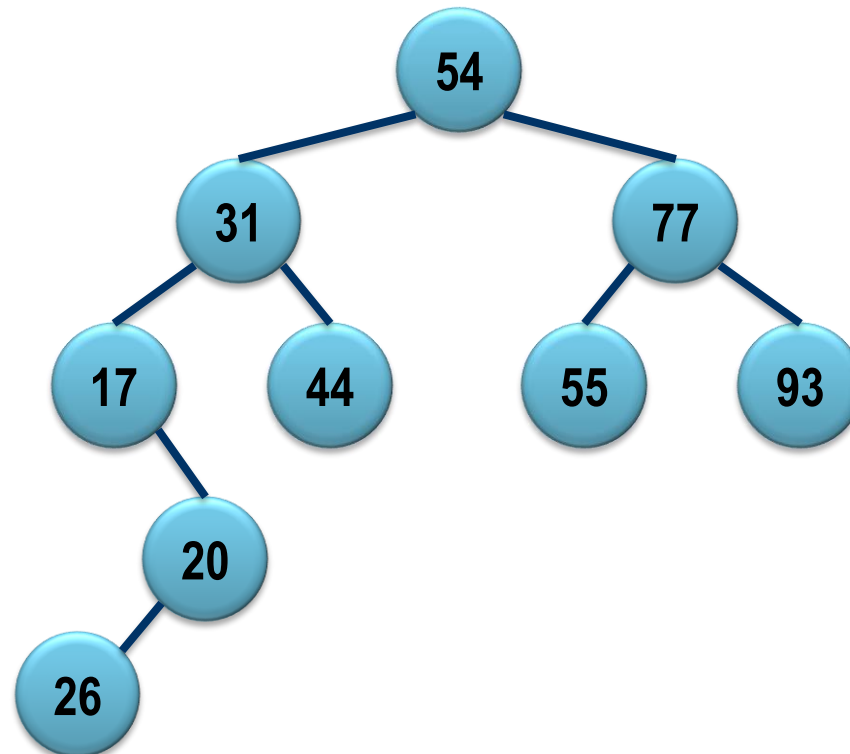




快速排序 - 举例

快速排序示例的递归树

17	20	26	31	44	54	55	77	93
----	----	----	----	----	----	----	----	----





快速排序 - 性能分析

- 对序列划分操作时间复杂度为 $O(n)$ ，最好情况下，每个次划分将序列分成两个大小相等的部分，这样时间复杂度为 $O(n \log_2 n)$
- 平均情况下，快速排序的时间复杂度也是 $O(n \log_2 n)$
- 对于已经有序的数据序列，所有的划分将退化（每次子序列规模减一），在这种最坏情况下，快速排序的时间复杂度为 $O(n^2)$
- 快速排序平均情况下需要 $O(\log_2 n)$ 的递归栈开销，但在最坏情况下可以达到 $O(n)$
- 快速排序是一种不稳定的排序方法



快速排序 - 改进

- 基本的快速排序算法选择最右（左）端元素为划分元素
- 当划分元素是序列中最大（最小）的元素，是划分的最坏情况
- 为了避免最坏情况，可采用：
 - **中间元素法**：取左端元素，中间元素和右端元素，选择关键字值处于中间的元素作为划分元素
 - **随机快速排序**：以等概率方式（人为引入随机性）随机选择一个划分元素（舍伍德算法）
- 这样可以避免最坏情况的发生，改善基本快速排序算法的性能



快速排序 - 改进

- 小序列仍需递归调用，增加计算复杂度
- 改进办法：
 - 当子序列规模过小，完成划分后，对整个序列调用插入排序
 - 此时插入排序接近最好情况 $O(n)$



快速排序 – 其他实现方式

20	26	93	17	77	31	44	55	54	
20	26	93	17	77	31	44	55		54
20	26	93	17	77	31	44	55		54
20	26		17	77	31	44	55	93	54
20	26		17	77	31	44	55	93	54
20	26	44	17	77	31		55	93	54
20	26	44	17		31	77	55	93	54
20	26	44	17	31		77	55	93	54
20	26	44	17	31	54	77	55	93	



提纲

- 基本概念
- 基本排序方法
- 希尔排序
- 快速排序
- 归并排序
- 堆排序



归并

- 归并：将两个或两个以上的有序序列合并成一个新的有序序列
- 二路归并：两个有序序列的保序归并

```
template<class T>void merge(T *c, T *a, int n, T *b, int m) {
```

```
    long i, j, k;
```

```
    for(i = 0, j = 0, k = 0; k < m+n; k++) {
```

```
        if(i == n) { c[k] = b[j]; j++; continue;}
```

```
        if(j == m) { c[k] = a[i]; i++; continue;}
```

```
        c[k] = (a[i] > b[j]) ? a[i++]: b[j++];
```

```
    }
```

```
}
```

- 若归并后序列长度为 n ，二路归并算法复杂度为 $O(n)$
- 同时需要 $O(n)$ 的存储空间

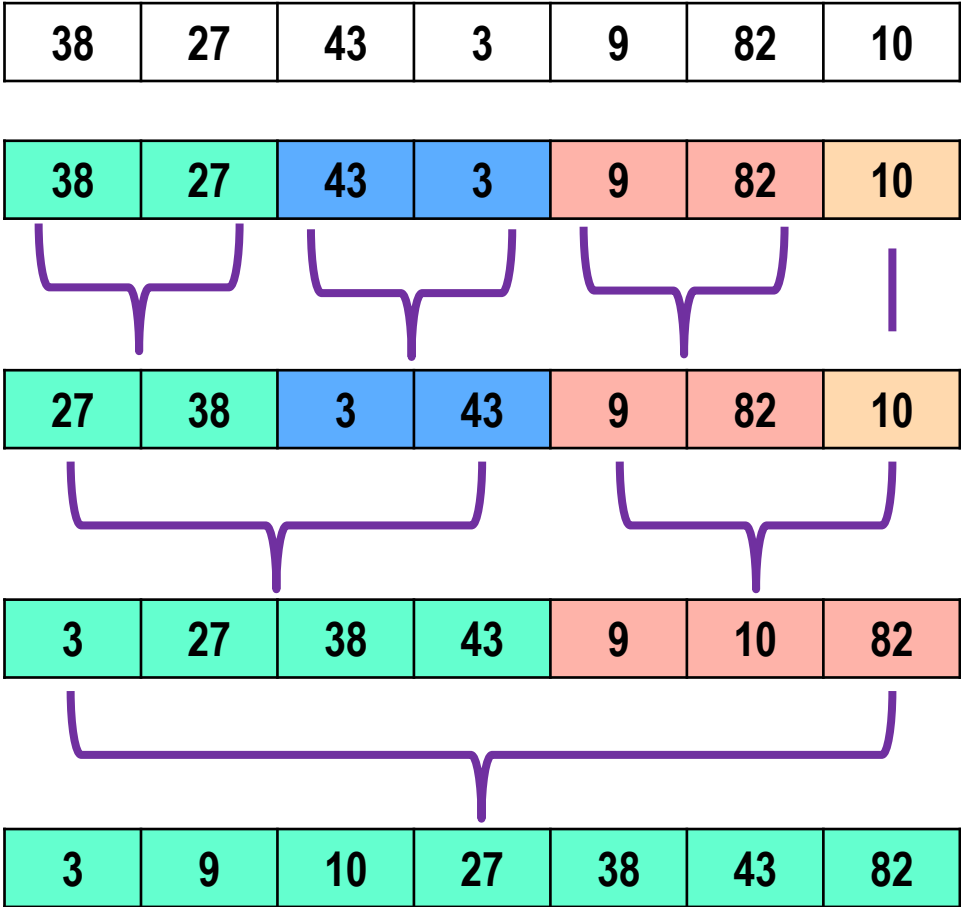


归并排序 - 自底向上

- 1945年，冯·诺依曼（J. V. Neumann）（42岁）发明了归并排序，最开始，归并排序采用的是自顶向下的模式，后来，到了1948年，冯和戈德斯坦（H. Goldstine）共同提出自底向上模式
- 把序列看成 n 个长度为1的有序子序列，自底向上，两两归并，最后得到长度为 n 的有序序列
- 自底向上归并排序中，子序列的大小是2的幂数，最后一个子序列例外



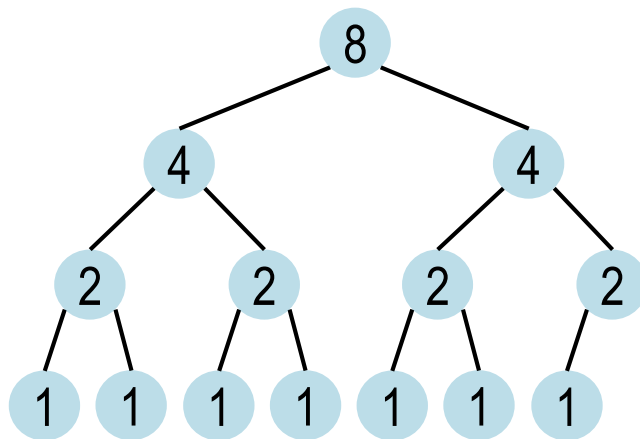
归并排序 - 自底向上 - 举例





归并排序 - 自底向上 - 举例

- 自底向上归并排序的归并树



- 完全二叉树，深度 $\lfloor \log_2 n \rfloor$



归并排序 - 自底向上

```
template<class T>void merge_sort(T *a, int l, int r){  
    int i, m;  
    for(m = 1; m <= r-l; m = m+m) {           //每次序列长度加倍  
        for(i = l; i <= r-m; i += m+m)        //归并每一对m长序列  
            merge(a,i,i+m, min(i+m+m-1,r));  
    } }
```

- 每一次归并的时间复杂度为 $O(n)$ ，次数为 $\lfloor \log_2 n \rfloor + 1$
- 自底向上归并排序算法的时间复杂度为 $O(n \log_2 n)$
- 缺点：占用内存较多，额外存储空间 $O(n)$
- 若归并算法是稳定的，则归并排序是稳定的排序方法



归并排序 - 自底向上

```
template<class T>void merge(T *c, T *a, int n, T *b, int m) {
```

```
    long i, j, k;
```

```
    for(i = 0, j = 0, k = 0; k < m+n; k++) {
```

```
        if(i == n) { c[k] = b[j]; j++; continue;}
```

```
        if(j == m) { c[k] = a[i]; i++; continue;}
```

```
        c[k] = (a[i] > b[j]) ? a[i++]: b[j++];
```

```
    }
```

```
}
```

43	38(A)	30	4	38(B)	10	2
----	-------	----	---	-------	----	---



43	38(B)	38(A)	30	10	4	2
----	-------	-------	----	----	---	---

➤ 若归并算法是稳定的，则归并排序是稳定的排序方法



归并排序 - 自顶向下

- 类似快速排序，归并排序也可利用划分子序列的方法递归实现
 - 首先把整个待排序序列划分为两个长度大致相等的部分，称为左子表和右子表
 - 分别对子表递归进行排序
 - 归并排序好的两个子表



归并排序 - 自顶向下 - 实现

```
template<class T>void merge_sort(T *a, int l, int r){  
    if(r <= l) return;           //子序列长度小于1  
    int m = (l + r)/2;           //求中点  
    merge_sort(a,l,m);           //对前半序列递归调用归并排序  
    merge_sort(a,m+1,r);         //对后半序列递归调用归并排序  
    merge(a,l,m,r);              //两个子序列的二路归并  
}
```

- 与自底向上相似，复杂度为 $O(n \log_2 n)$ ，需要 $O(n)$ 的辅助空间
- 运行时间只与序列长度有关，与输入数据的序列特性无关，不会出现急剧恶化的情况 → **快速排序**



提纲

- 基本概念
- 基本排序方法
- 希尔排序
- 快速排序
- 归并排序
- 堆排序



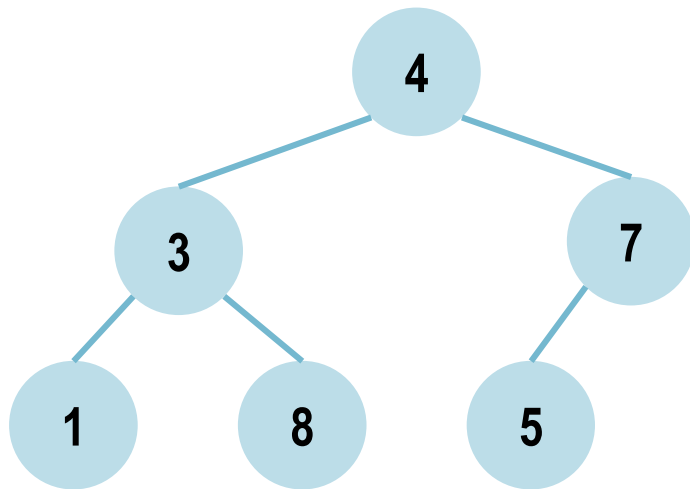
堆排序

- 最大堆是一棵**完全二叉树**，并且堆中任一结点的值都小于其父结点的值，**堆顶结点的值最大（部分有序）**
- 堆排序：从选择排序的思路出发，利用堆的结构实现排序
 - 选择排序：实现简单，效率低
 - 利用堆实现：具有树结构选择排序效率高，又可以**原地工作**，空间效率高
- 操作步骤
 - 构造初始的最大堆；
 - 堆尾和堆顶元素交换位置，堆序列长度减一，调整堆
 - 重复这个过程，当堆为空时，排序完成



堆排序 - 举例

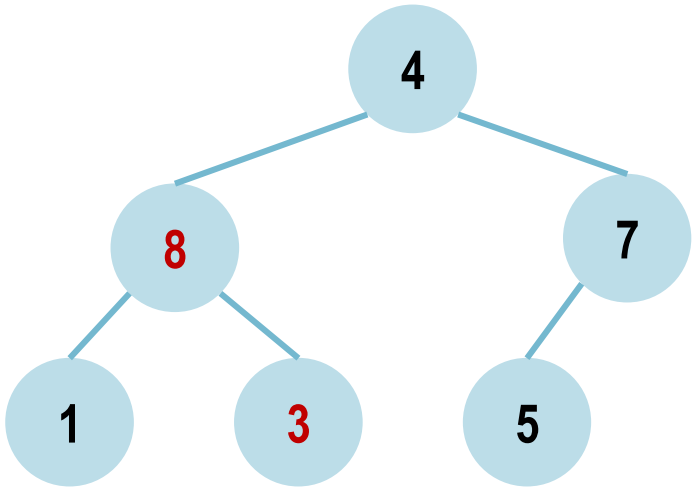
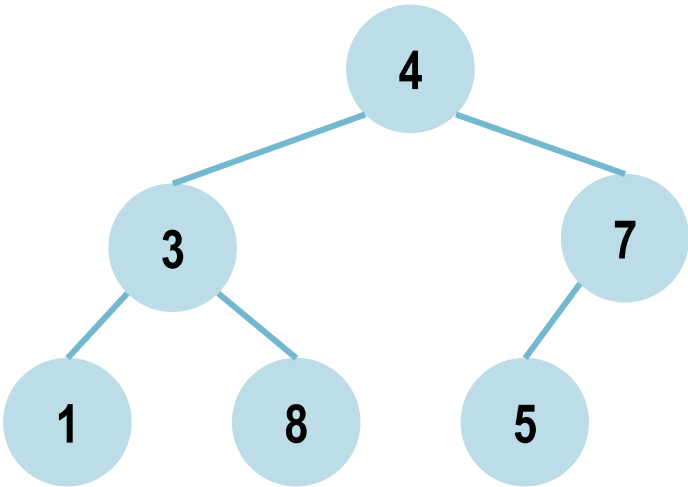
4	3	7	1	8	5
---	---	---	---	---	---





堆排序 - 举例 - 自底向上构造最大堆

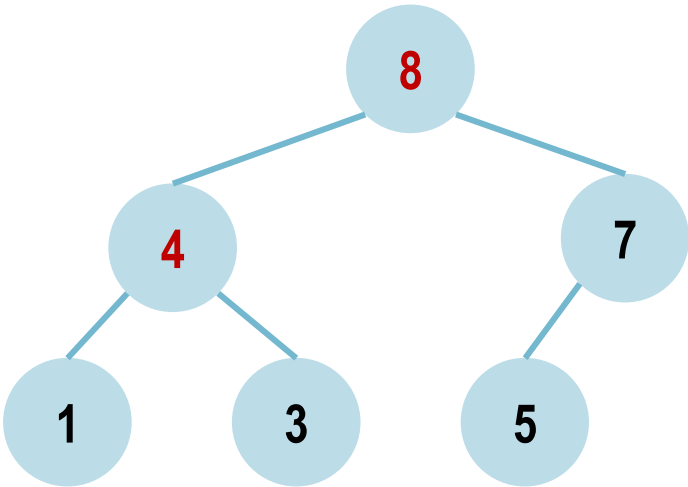
4	3	7	1	8	5
4	8	7	1	3	5





堆排序 - 举例 - 自底向上构造最大堆

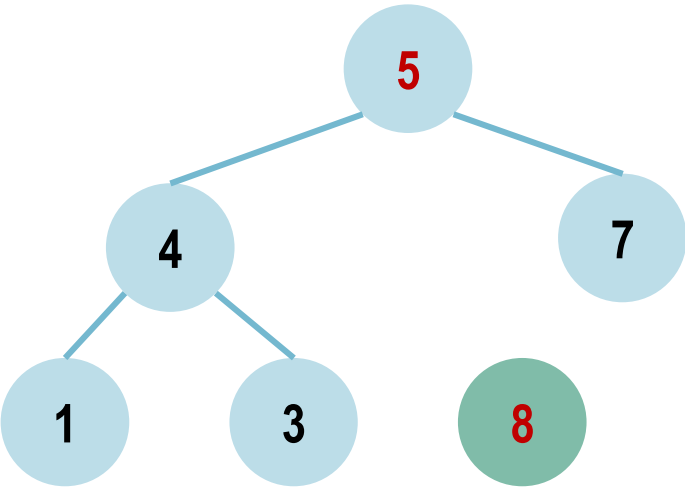
4	3	7	1	8	5
8	4	7	1	3	5





堆排序 - 举例 - 取堆顶元素

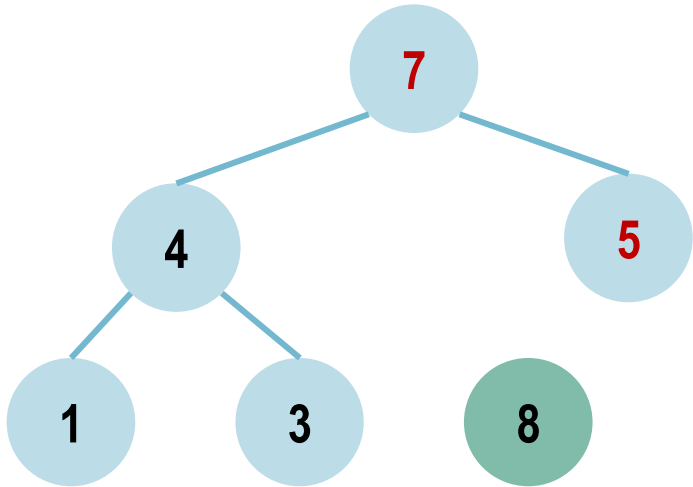
4	3	7	1	8	5
5	4	7	1	3	8





堆排序 - 举例 - 堆化

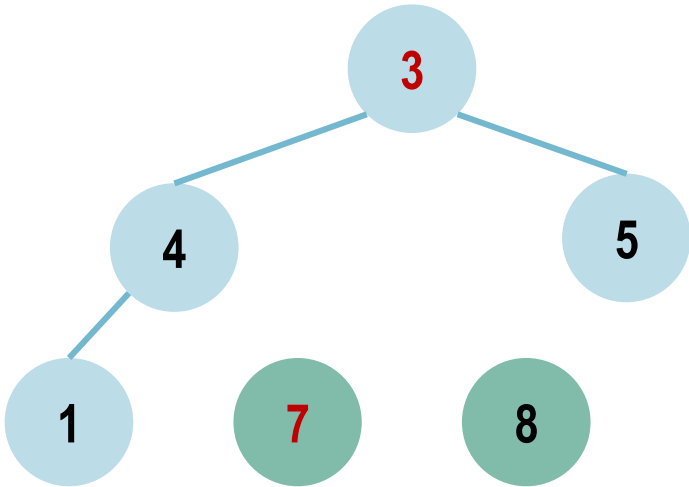
4	3	7	1	8	5
7	4	5	1	3	8





堆排序 - 举例 - 取堆顶元素

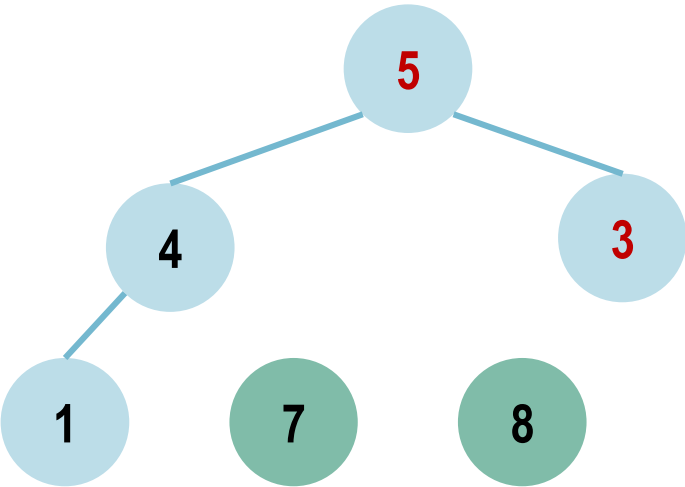
4	3	7	1	8	5
3	4	5	1	7	8





堆排序 - 举例 - 堆化

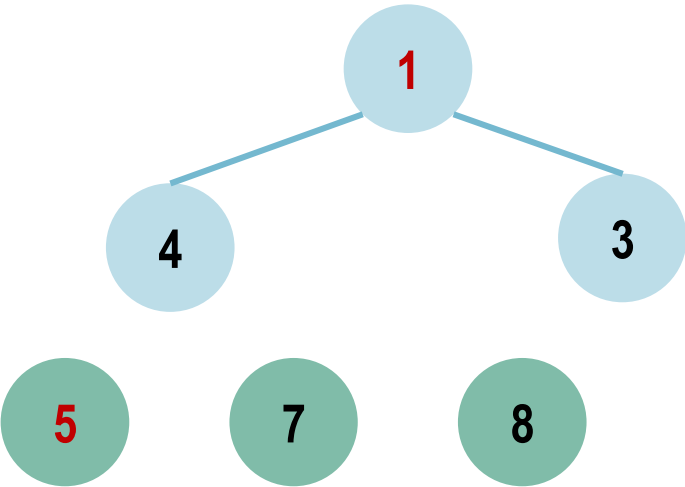
4	3	7	1	8	5
5	4	3	1	7	8





堆排序 - 举例 - 取堆顶元素

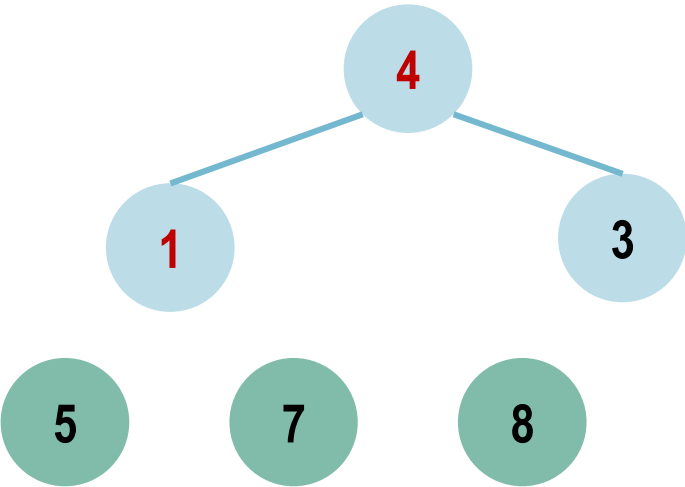
4	3	7	1	8	5
1	4	3	5	7	8





堆排序 - 举例 - 堆化

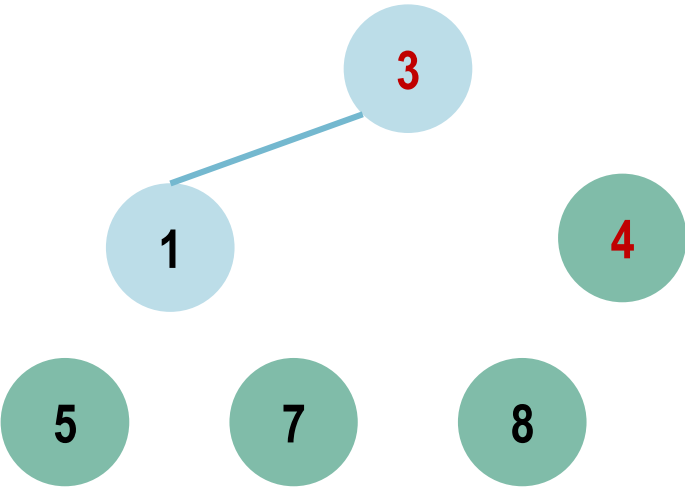
4	3	7	1	8	5
4	1	3	5	7	8





堆排序 - 举例 - 取堆顶元素

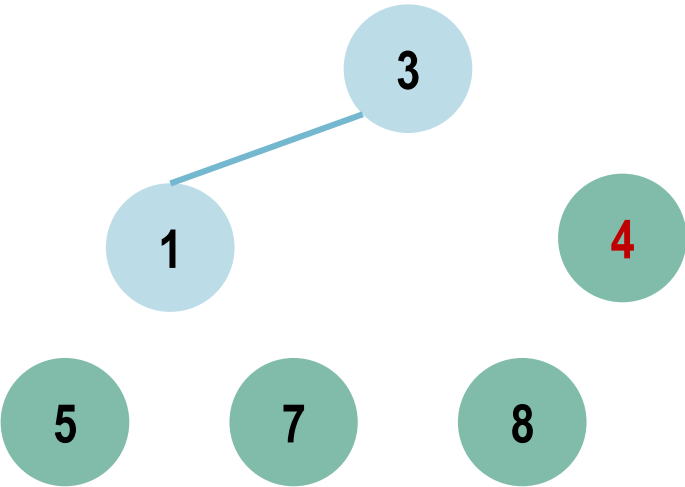
4	3	7	1	8	5
3	1	4	5	7	8





堆排序 - 举例 - 堆化

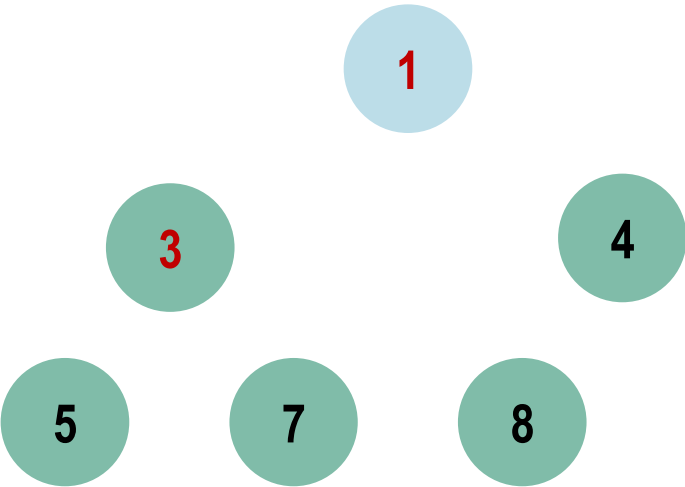
4	3	7	1	8	5
3	1	4	5	7	8





堆排序 - 举例 - 取堆顶元素

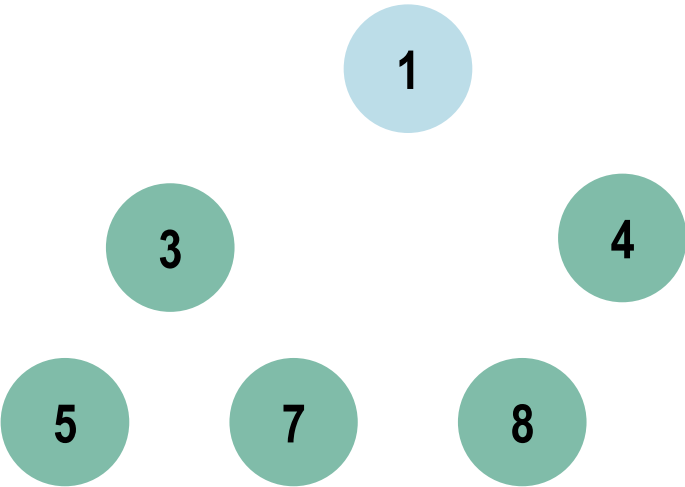
4	3	7	1	8	5
1	3	4	5	7	8





堆排序 - 举例 - 堆化

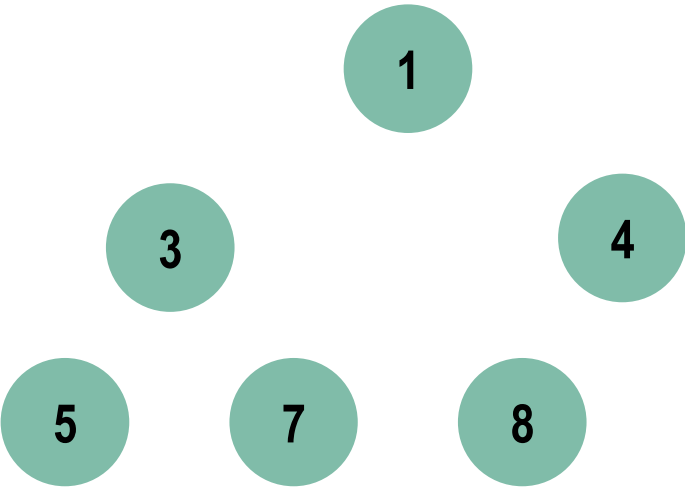
4	3	7	1	8	5
1	3	4	5	7	8





堆排序 - 举例 - 取堆顶元素

4	3	7	1	8	5
1	3	4	5	7	8





堆排序 - 实现

```
template<class T>void heap_sort(T *a, int l, int r){  
    int k, N = r-l;                                //堆的大小  
    T *p = a + l;  
    for(k = (N-1)/2; k >= 0; k--)                  //自底向上构造堆  
        fixDown(p, k, N);                          //自顶向下堆化  
    while(N > 0){  
        exch(p[0],p[N]);                          //堆顶和堆尾元素交换  
        N--;                                       //堆规模减一  
        fixDown(p,0,N);                          //调整堆  
    }  
}
```



堆排序 - 复杂度分析

➤ 时间复杂度

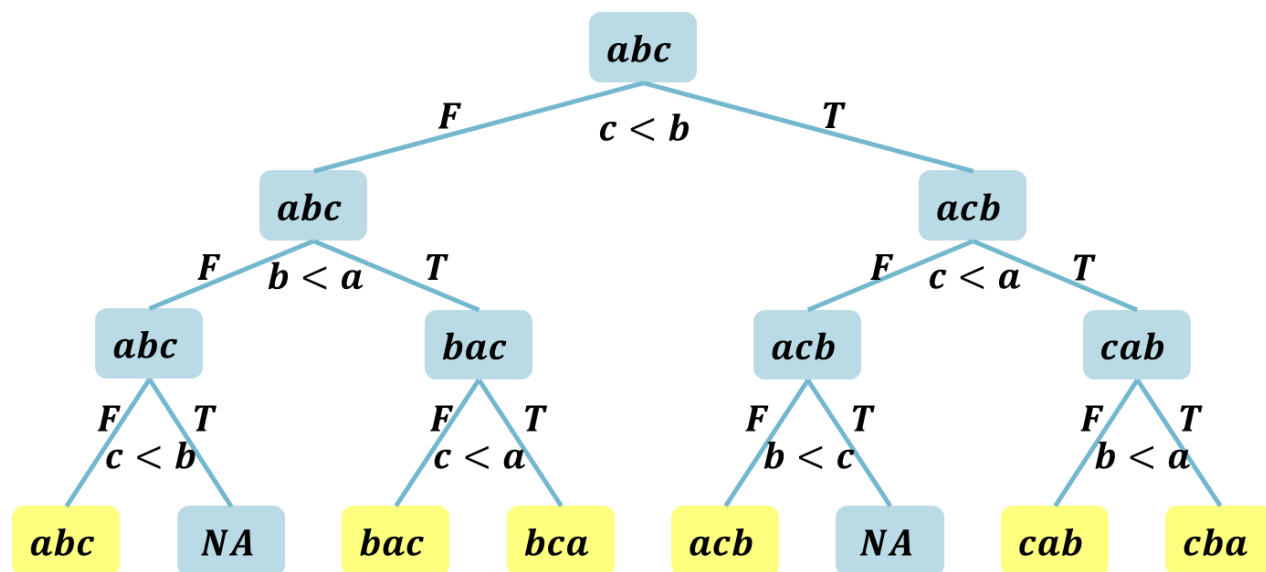
- 自底向上构造堆的时间复杂度： $O(n)$
- 删除堆顶元素（ n 个）并重新调整为新堆的时间复杂度： $O(\log_2 n)$
- 时间复杂度： $O(n \log_2 n)$

➤ 原地操作：不需要额外的空间

➤ 不稳定的排序方法



基于比较排序的决策树表示



决策树描述一种排序算法的过程，序列初始顺序决定算法在决策树上经过的路径

- 非叶子结点代表一次比较操作，两个分支描述两种比较结果
- 叶子结点代表排序结果，所有叶子结点包含了所有可能的排序结果



基于比较排序方法的下界分析

- n 元序列共 $n!$ 种排列方式
 - 决策树叶子结点总数大于等于 $n!$
 - 若叶子结点数目大于 $n!$ ，一定存在无效结点或者重复结点
- 用决策树来描述一个 n 元序列的排序问题
 - 高度为 i 的二叉决策树最多可能有 2^{i-1} 叶子结点
 - $2^{i-1} \geq n!$
 - $i - 1 \geq O(\log_2(n!)) \Rightarrow i \geq O(n \log_2 n)$
 - 决策树高度 i 确定了需要进行比较的次数
- 一般情况下排序算法可期待的最好的时间复杂度为 $O(n \log_2 n)$

排序方法小结

➤ 冒泡排序

➤ 插入排序

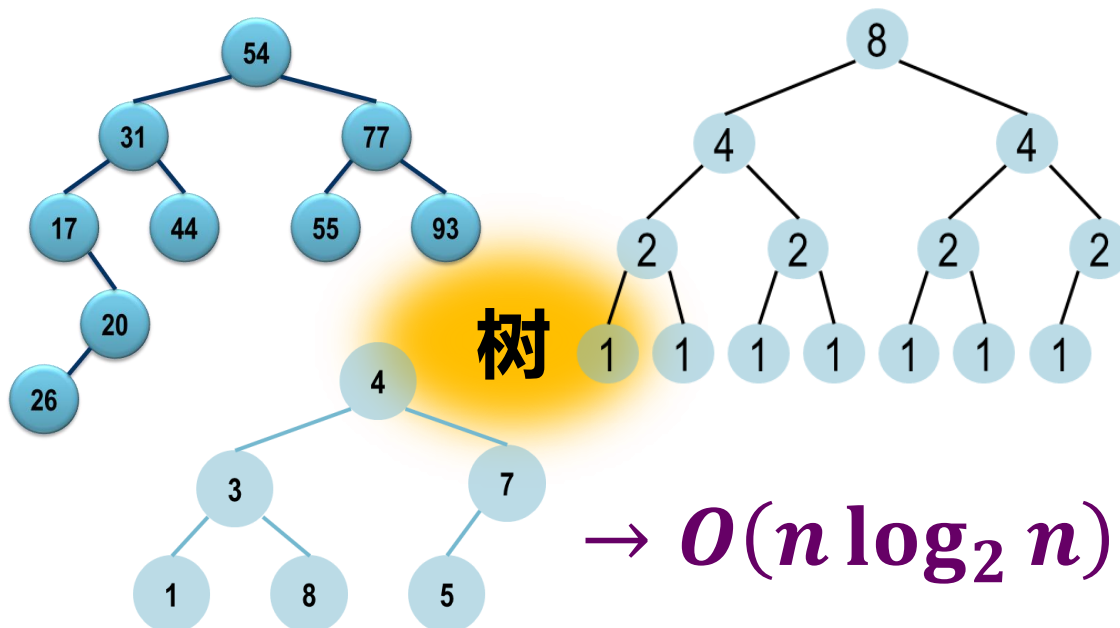
➤ 选择排序

平均时间复杂度都是 $O(n^2)$

➤ 快速排序

➤ 归并排序

➤ 堆排序





排序方法小结

排序方法	比较次数		移动次数		稳定性	附加存储	
	最好情况	最坏情况	最好情况	最坏情况		最好	最差
直接插入排序	$O(n)$	$O(n^2)$	$O(n)$	$O(n^2)$	√	1	
折半插入排序	$O(n \log_2 n)$		$O(n)$	$O(n^2)$	√	1	
冒泡排序	$O(n)$	$O(n^2)$	0	$O(n^2)$	√	1	
选择排序	$O(n^2)$		0	$O(n)$	×	1	
快速排序	$O(n \log_2 n)$	$O(n^2)$	$O(n \log_2 n)$	$O(n^2)$	×	$O(\log_2 n)$	$O(n)$
归并排序	$O(n \log_2 n)$		$O(n \log_2 n)$		√	$O(n)$	

- 平均性能：快速排序优于归并排序
- 快速排序不稳定，且性能可能退化

加油同学们!

